B.C.A. Semester V · Skill Enhancement Course (SEC)

ANDROID PROGRAMMING

Jaipur National University, Jaipur

Paper Code: BCAAPCO505T22 | Official SEC Syllabus

Exhaustive Topper Notes · Java OOP + Eclipse ADT + Activity Lifecycle + Widgets + SQLite

Unit I: Intro & Architecture

Unit II: Java OOP & JVM

Unit III: Eclipse ADT & Deployment

Unit IV: UI Arch & Widgets

Unit V: SQLite Database

Syllabus Environment: Java, Eclipse with ADT, Android Emulator, Android Sandwich/Jelly Bean & USB Debugging

📑 Table of Contents

1. **Unit I: Introduction & Android Architecture**
   - History of Android & Version Timeline
   - Android Operating System Overview & Features
   - Android Development Tools & ADB Commands
   - Android Architecture Deep-Dive (5 Layers)
2. **Unit II: Object Oriented Programming Using Java**
   - OOP Principles, Modifiers & Class/Object
   - Inheritance & Polymorphism
   - Interfaces & Abstract Classes
   - Multithreading & Thread Synchronization
   - JVM Architecture & Garbage Collection
   - 7 Essential Exam Java Programs with Explanations
3. **Unit III: Development Tools & Environment**
   - Installing & Using Eclipse with ADT Plugin
   - Android Virtual Device (AVD / Emulator)
   - AndroidManifest.xml & Project Structure
   - Creating Project & Hello World Code
   - Deploying on USB-Connected Device
4. **Unit IV: User Interface Architecture and Design**
   - Application Context & Intent Passing
   - Activity Life Cycle (7 States & Transitions)
   - Screen Sizes, Densities & dp/sp Math
   - Form Widgets, Input Controls & Layout Managers
   - Menus, Dialogs & Toast Notifications
5. **Unit V: Database (SQLite)**
   - Understanding SQLite & Supported Data Types
   - Connecting via SQLiteOpenHelper
   - Complete CRUD Operations Java Code
   - App to Database Architecture Flow
6. 📖 **Important Definitions**
7. ⚖️ **Important Comparison Tables**
8. 🎯 **Master Revision**
9. 📅 **One-Day Revision Sheet**
10. ❓ **Exam Question Bank**

11. 📝 **Topper Exam-Writing Strategy**

I Unit I: Introduction & Android Architecture

Unit Overview:

- Covers History of Android, version evolution, Android OS features, SDK tools (ADB, DDMS), and the 5-layer Android Architecture.
- Key Exam Drivers: 🔥 5-Layer Android Architecture Diagram, Features of Android OS, and ADT/SDK tools.

1.1 History of Android & Version Timeline ☆ Important

HISTORY SUMMARY

Android was founded in October 2003 by **Andy Rubin**, Rich Miner, Nick Sears, and Chris White. It was acquired by **Google** in 2005. In November 2007, the **Open Handset Alliance (OHA)**—a consortium of 84 hardware, software, and telecom companies—was launched to establish open standards for mobile devices.

Android Version Timeline (Syllabus Context):

Version	Code Name	API Level	Key Innovations
1.5	Cupcake	API 3	On-screen virtual keyboard, video recording, widgets.
1.6	Donut	API 4	Quick Search Box, support for multiple screen resolutions (WVGA).
2.0 / 2.1	Eclair	API 5-7	Google Maps Navigation, HTML5 support, live wallpapers.
2.2	Froyo	API 8	Speed optimizations via Dalvik JIT, USB tethering, Wi-Fi hotspot.
2.3	Gingerbread	API 9-10	NFC support, enhanced copy-paste, multiple camera support.
3.0 - 3.2	Honeycomb	API 11-13	Tablet-only UI, Action Bar, multi-core processor optimization.
4.0	Ice Cream Sandwich (ICS)	API 14-15	Unified Smartphone & Tablet UI, resizable widgets, Face Unlock.
4.1 - 4.3	**Jelly Bean**	**API 16-18**	**Project Butter (60fps UI), Google Now, expandable notifications (Primary Syllabus Baseline).**

1.2 Android Operating System Overview & Features 🔥 Very Important

> DEFINITION
>
> **Android** is an open-source, Linux-based mobile operating system designed primarily for touchscreen mobile devices such as smartphones and tablets.

Key Features of Android OS:

- **Open Source & Customizability:** Source code is released under the Apache 2.0 open-source license.
- **Multitasking & Process Management:** Runs multiple applications concurrently while automatically managing background memory.
- **SQLite Relational Database:** Embedded lightweight database engine for structured local persistent storage.

- **Comprehensive Connectivity:** Built-in support for GSM/EDGE, IDEN, CDMA, EV-DO, UMTS, LTE, Wi-Fi, Bluetooth, NFC, and GPS.
- **Hardware Integration:** Direct API integration with camera, touch sensors, accelerometer, gyroscope, proximity sensor, and GPS.
- **Dalvik Virtual Machine (DVM):** Execution engine optimized specifically for small memory footprints.
- **Rich Media & Graphic Support:** Supports WebGL, OpenGL ES 2.0/3.0, PNG, JPEG, GIF, MP3, MP4, AAC, and H.264 video.

1.3 Android Development Tools & ADB Commands ☆ Important

Tool Name	Purpose	Key Functionality
Eclipse IDE	Code Editor	Primary Java and XML development IDE specified in syllabus.
ADT Plugin	Eclipse Extension	Provides Android project creation, UI layout previews, and build automation.
Android SDK	API Libraries	Contains Android class libraries, compilers (`dx` tool), and platform headers.
AVD Manager	Emulator Manager	Creates virtual phone instances with custom RAM, screen resolution, and API targets.
ADB (Android Debug Bridge)	CLI Bridge Utility	Manages device connection, app installation, and shell execution.
DDMS	Monitoring Server	Monitors process heap, threads, logcat outputs, and screen capture.

Essential ADB Command Summary (Exam Favorite):

```
# List all connected physical devices and emulators
adb devices

# Install an APK onto connected device
adb install MyApp.apk

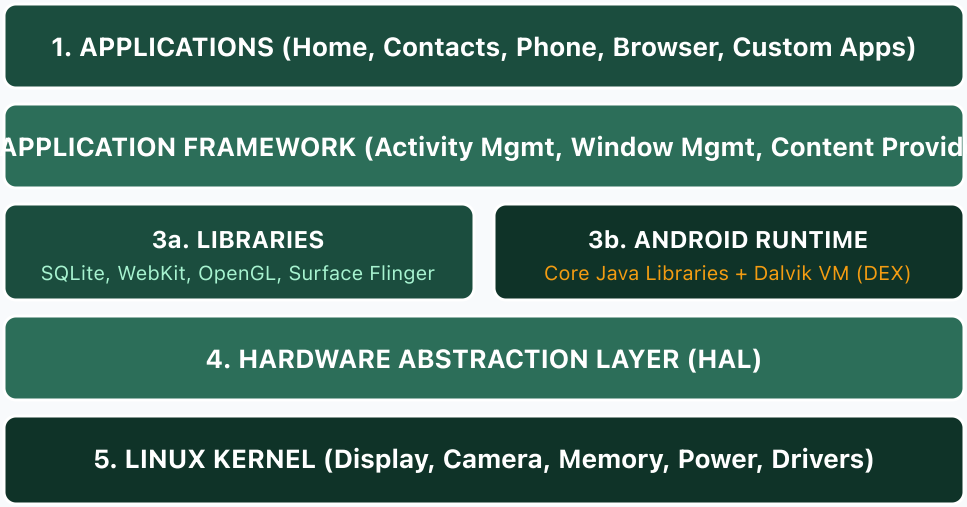
# View real-time system logs
adb logcat

# Open a Linux command shell on target device
adb shell
```

1.4 Android Architecture Deep-Dive (5 Layers) 🔥 Very Important

DEFINITION

The **Android Architecture** is structured as a software stack composed of 5 distinct layers operating on top of a customized Linux Kernel.



Exam Drawing Tip: Draw 5 horizontal stacked boxes. Top: Applications → App Framework → C/C++ Libraries & Android Runtime (side-by-side) → HAL → Bottom: Linux Kernel.

Exhaustive Layer Breakdown:

1. **Applications Layer:** Pre-installed native apps (Contacts, Phone, Gallery) and third-party user applications built using the Java API.
2. **Application Framework Layer:** High-level Java services used by developers:
   - *Activity Manager:* Manages activity lifecycle and backstack.
   - *Window Manager:* Manages screen display windows.
   - *Content Providers:* Enables data sharing across applications.
   - *Resource Manager:* Grants access to non-code assets (layouts, strings, icons).
   - *Notification Manager:* Displays alerts and status bar notifications.
3. **C/C++ Native Libraries Layer:** Low-level C/C++ compiled binaries:
   - *Surface Flinger:* Composites 2D/3D graphics onto screen frame buffer.
   - *SQLite:* Lightweight relational database engine.
   - *WebKit:* Browser engine for rendering web pages.
   - *OpenGL ES:* 3D graphics rendering API.
   - *Media Framework:* Audio/video playback and recording.
4. **Android Runtime (ART / Dalvik):** Includes core Java libraries and the **Dalvik Virtual Machine (DVM)**. DVM converts Java bytecode (`.class`) into Dalvik Executable (`.dex`) format using the `dx` tool, optimizing memory consumption.
5. **Hardware Abstraction Layer (HAL):** Standardized interface modules separating high-level framework Java calls from underlying hardware drivers.
6. **Linux Kernel Layer:** Base operating system layer managing process creation, low-memory killer (LMK), binder IPC driver, wakelocks (power management), memory allocation, and hardware device drivers (Camera, Display, Wi-Fi, Bluetooth).

📝 10-MARK ANSWER: EXPLAIN THE ANDROID ARCHITECTURE IN DETAIL WITH A NEAT LAYERED DIAGRAM.

1. Introduction: Android is built as a stacked software platform running on top of a customized Linux Kernel.

2. Diagram: Draw the 5-layer Android Architecture diagram shown above.

3. Layer Explanations: Detail Applications, Application Framework, C/C++ Libraries, Dalvik Runtime, HAL, and Linux Kernel.

4. Dalvik VM Special Role: Explain how Java bytecode (`.class`) is converted to DEX bytecode (`.dex`) using the `dx` compiler tool for low memory execution.

5. Conclusion: The layered architecture provides modularity, hardware independence, and process isolation across diverse device manufacturers.

HOW TO WRITE IN EXAM: Start with Android definition → Draw 5-layer diagram → Detail each layer with 3 bullet points → Add Dalvik VM box → Conclude with architecture benefits.

📌 Quick Revision — Unit I

Android: Open-source, Linux-based mobile OS created by OHA / Google.	5 Architecture Layers: Applications, Framework, Libraries/Runtime, HAL, Kernel.
Linux Kernel: Handles memory, security, process & hardware drivers.	Dalvik VM: Executes .dex files optimized for low RAM devices.
ADT Plugin: Eclipse extension for building Android apps.	ADB Commands: adb devices, adb install, adb logcat, adb shell.

🧠 MEMORY TRICKS — UNIT I

- Android Architecture Layers: **A-F-R-L-K** → **A**pplications, **F**ramework, **R**untime, **L**ibraries, **K**ernel.
- Dev Tools: **E-A-S-A-D** → **E**clipse, **A**DT, **S**DK, **A**VD, **D**DMS.

II Unit II: Object Oriented Programming Using Java

Unit Overview:

- Covers Java OOP concepts (Inheritance, Polymorphism, Interfaces, Abstract Classes, Overloading/Overriding), Multithreading & Synchronization, JVM Architecture & Garbage Collection, and 7 Exam Java programs.
- Key Exam Drivers: 🔥 Overloading vs Overriding, Interface vs Abstract Class, Thread Lifecycle, JVM Architecture, and Practical Java Programs.

2.1 Overview of OOP Concepts & Access Modifiers ☆ Important

- **Class:** A blueprint or prototype containing data fields and member methods.
- **Object:** A concrete instance of a class allocated in Heap memory.
- **Encapsulation:** Wrapping state variables and behavior methods inside a single unit (class) while using `private` modifiers to restrict direct access.
- **Abstraction:** Exposing only essential interface features while concealing internal implementation details.

Java Access Modifiers Summary Table:

Modifier	Same Class	Same Package	Subclass (Different Pkg)	World (Anywhere)
`private`	Yes	No	No	No
`default` (none)	Yes	Yes	No	No
`protected`	Yes	Yes	Yes	No
`public`	Yes	Yes	Yes	Yes

2.2 Inheritance & Polymorphism 🔥 Very Important

> **INHERITANCE**
>
> The mechanism where a child class (subclass) derives properties and methods from a parent class (superclass) using the `extends` keyword, enabling code reusability.

Types of Inheritance: Single, Multilevel, Hierarchical (Multiple inheritance of classes is prohibited in Java to avoid ambiguity; implemented via *Interfaces*).

> **POLYMORPHISM**
>
> The ability of an object or method to take on multiple forms:
> 1. **Compile-Time Polymorphism (Method Overloading / Static Binding)**
> 2. **Runtime Polymorphism (Method Overriding / Dynamic Binding)**

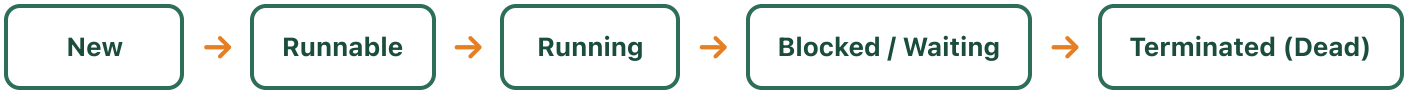
2.3 Interfaces & Abstract Classes 🔥 Very Important

Feature	Abstract Class	Interface
Keyword	Declared using `abstract class`.	Declared using `interface`.
Methods	Can contain abstract (no body) and concrete methods.	Contains default 100% abstract methods.
Multiple Inheritance	Does not support multiple class inheritance.	Supports multiple inheritance (a class can `implements` multiple interfaces).
Variables	Can have final, non-final, static, and non-static variables.	Variables are implicitly `public static final` (constants).
Implementation	Extended using `extends`.	Implemented using `implements`.

2.4 Multithreading & Thread Synchronization 🔥 Very Important

> **THREAD DEFINITION**
>
> A **Thread** is a lightweight sub-process executing concurrently within a Java application context.

Thread Life Cycle (5 States):



Thread Synchronization: Using the `synchronized` keyword prevents thread interference and concurrency errors when multiple threads access shared resources.

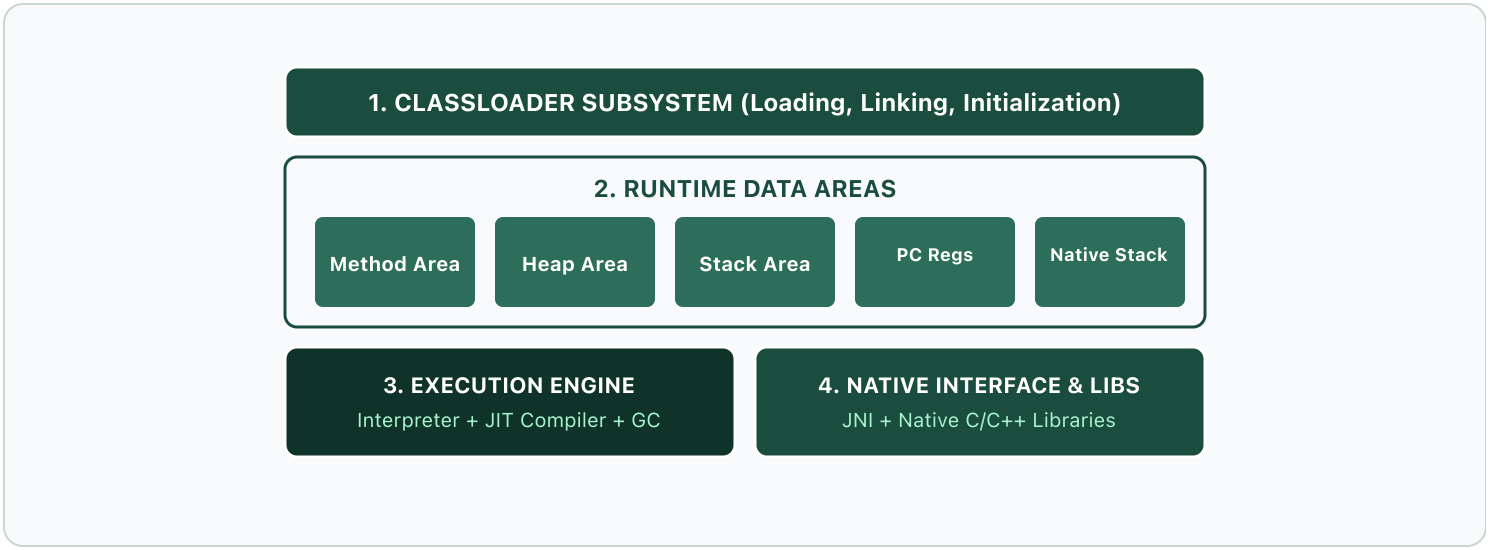
2.5 Method Overloading vs. Method Overriding 🔥 Very Important

Feature	Method Overloading	Method Overriding
Definition	Multiple methods in same class with same name but different parameters.	Subclass provides specific implementation of method defined in superclass.
Location	Occurs within the *same class*.	Occurs between *two classes* having inheritance relationship.
Parameters	Must be *different* (type, number, or order).	Must be *exact same* signature.
Binding	Compile-time (Static binding).	Runtime (Dynamic binding).
Return Type	Can be same or different.	Must be same (or covariant).

2.6 JVM Architecture & Garbage Collection 🔥 Very Important

> JVM DEFINITION
>
> **JVM (Java Virtual Machine)** is an abstract virtual execution engine that converts Java bytecode (`.class` files) into native machine code for execution on physical hardware.





Garbage Collection (GC): Automatic memory management module in JVM that reclaims Heap memory occupied by unreferenced objects using *Mark and Sweep* algorithms.

2.7 Essential Exam Java Programs 🔥 Very Important

Program 1: Class, Object & Constructor

```
class Student {
    int id;
    String name;
    Student(int i, String n) { id = i; name = n; }
    void display() { System.out.println("ID: " + id + ", Name: " + name); }

    public static void main(String[] args) {
        Student s1 = new Student(101, "Rahul");
        s1.display();
    }
}
```

```
OUTPUT:
ID: 101, Name: Rahul
```

Program 2: Single Inheritance

```
class Animal {
    void eat() { System.out.println("Eating..."); }
}
class Dog extends Animal {
    void bark() { System.out.println("Barking..."); }

    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}
```

OUTPUT:

```
Eating...
Barking...
```

Program 3: Method Overloading

```
class Calculator {
    int add(int a, int b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }

    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println("Sum 2: " + calc.add(5, 10));
        System.out.println("Sum 3: " + calc.add(5, 10, 15));
    }
}
```

OUTPUT:

```
Sum 2: 15
Sum 3: 30
```

Program 4: Method Overriding

```
class Bank {
    int getRateOfInterest() { return 0; }
}
class SBI extends Bank {
    @Override
    int getRateOfInterest() { return 7; }

    public static void main(String[] args) {
        Bank b = new SBI();
        System.out.println("SBI Interest Rate: " + b.getRateOfInterest() + "%");
    }
}
```

OUTPUT:

```
SBI Interest Rate: 7%
```

Program 5: Interface Implementation

```
interface Printable {
    void print();
}
class Document implements Printable {
    public void print() { System.out.println("Printing Document..."); }

    public static void main(String[] args) {
        Printable doc = new Document();
        doc.print();
    }
}
```

OUTPUT:

```
Printing Document...
```

Program 6: Abstract Class

```
abstract class Shape {
    abstract void draw();
}
class Rectangle extends Shape {
    void draw() { System.out.println("Drawing Rectangle..."); }

    public static void main(String[] args) {
        Shape s = new Rectangle();
        s.draw();
    }
}
```

OUTPUT:

```
Drawing Rectangle...
```

Program 7: Thread Creation using Runnable Interface

```
class MyThread implements Runnable {
    public void run() {
        System.out.println("Thread is running concurrently...");
    }

    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        Thread thread = new Thread(t1);
        thread.start();
    }
}
```

OUTPUT:

```
Thread is running concurrently...
```

📝 **10-MARK ANSWER: EXPLAIN JVM ARCHITECTURE IN DETAIL WITH COMPILATION PIPELINE AND EXECUTION SUBSYSTEMS.**

1. **Introduction:** JVM is the heart of Java's "Write Once, Run Anywhere" (WORA) philosophy.
2. **Compilation Flow:** Java Source (.java) → javac → Bytecode (.class) → JVM → Native OS Code.
3. **Diagram:** Draw 4-component JVM Diagram (ClassLoader, Memory, Engine, Native Interface).
4. **Subsystem Breakdown:** Detail ClassLoader (Load/Link/Init), Runtime Memory (Heap, Stack, Method area, PC, Native Stack), Execution Engine (Interpreter, JIT, Garbage Collector).
5. **Conclusion:** JVM provides platform independence by isolating execution from hardware differences.

HOW TO WRITE IN EXAM: Start with WORA concept → Draw compilation pipeline → Draw JVM diagram → Explain 5 runtime memory areas in bullets → Conclude.

📌 Quick Revision — Unit II

Encapsulation = Wrapping variables & methods into single class.

Inheritance = Subclass reuses fields/methods of superclass using extends.

Overloading = Same method name, different parameters in same class.

Overriding = Subclass redefines superclass method with same signature.

Interface = 100% abstract specification implemented via implements.

JVM Memory: Heap (Objects), Stack (Frame/local vars), Method Area (Classes).

🧠 MEMORY TRICKS — UNIT II

- JVM Memory Areas: **M-H-S-P-N** → **M**ethod, **H**eap, **S**tack, **P**C Register, **N**ative Stack.
- Thread Lifecycle: **N-R-R-B-T** → **N**ew, **R**unnable, **R**unning, **B**locked, **T**erminated.

III Unit III: Development Tools & Environment

Unit Overview:

- Covers setup and usage of Eclipse with ADT Plugin, Android Virtual Device (AVD / Emulator), Android Project Structure, Hello World application, and USB Device Deployment.
- Key Exam Drivers: 🔥 Android Development Workflow, Eclipse ADT project structure, AVD Emulator, and USB Debugging steps.

3.1 Installing & Using Eclipse with ADT Plugin 🔥 Very Important

SYLLABUS ENVIRONMENT SETUP

The official syllabus specifies developing Android applications using **Eclipse IDE** configured with the **Android Development Tools (ADT) Plugin**.

Installation & Configuration Steps:

1. Install Java Development Kit (JDK 6/7).
2. Download and extract Eclipse IDE for Java Developers.
3. Install ADT Plugin in Eclipse via `Help → Install New Software` using URL: `https://dl-ssl.google.com/android/eclipse/`.
4. Download Android SDK Manager and install SDK platforms (Android 4.1/4.2 Jelly Bean).
5. Configure SDK location in Eclipse via `Window → Preferences → Android → SDK Location`.

3.2 Android Virtual Device (AVD / Emulator) ☆ Important

DEFINITION — AVD

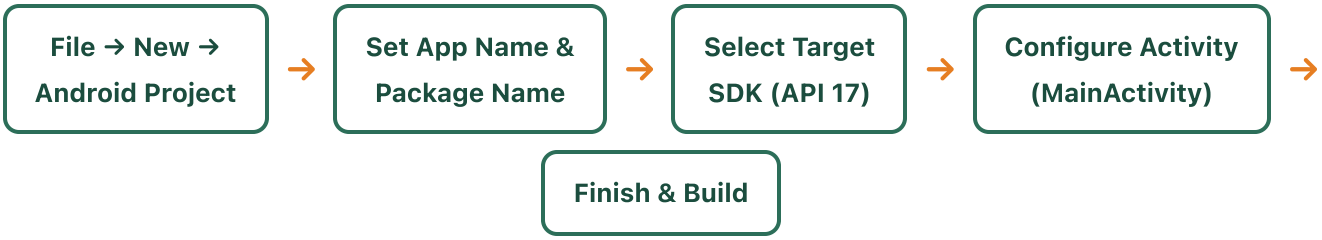
An **Android Virtual Device (AVD)** is an emulator configuration that simulates a physical Android device on the developer's computer to test apps without needing hardware.

Configuring an AVD in Eclipse:

- Open `Window → Android Virtual Device Manager`.

- Click **New** and specify: Device Name, Target API Level (e.g., API 17 Jelly Bean), CPU Architecture (ARM/x86), SD Card size, and RAM (e.g., 512 MB).
- Click **Create AVD** and launch the emulator.

3.3 AndroidManifest.xml & Project Structure 🔥 Very Important



Key Files in Android Project Structure:

Directory / File	Purpose & Description
`src/`	Contains Java source code files (e.g., `MainActivity.java`).
`gen/R.java`	Auto-generated file containing resource IDs; **never modify manually**.
`res/layout/`	Contains XML UI layout files (e.g., `activity_main.xml`).
`res/drawable-hdpi/`	Contains image assets (PNG, JPG) scaled for high-density screens.
`res/values/`	Contains `strings.xml`, `colors.xml`, and styles.
`AndroidManifest.xml`	Core manifest configuration file listing permissions, activities, and SDK versions.

Deep Breakdown of AndroidManifest.xml Tags:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.helloworld"
    android:versionCode="1"
    android:versionName="1.0" >

    <uses-sdk
        android:minSdkVersion="14"
        android:targetSdkVersion="17" />

    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name" >

        <activity
            android:name=".MainActivity"
            android:label="@string/app_name" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

    </application>
</manifest>
```

3.4 Creating Hello World Code Example 🔥 Very Important

1. XML Layout (`res/layout/activity_main.xml`):

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >

    <TextView
        android:id="@+id/txtHello"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerInParent="true"
        android:text="Hello World from Android!"
        android:textSize="20sp" />

</RelativeLayout>
```

2. Java Activity (`src/com/example/helloworld/MainActivity.java`):

```
package com.example.helloworld;

import android.app.Activity;
import android.os.Bundle;

public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

3.5 Deploying Application on USB-Connected Device 🔥 Very Important

Step-by-Step USB Deployment Procedure:

1. **Enable USB Debugging on Phone:** Open phone `Settings → Developer Options → Check "USB Debugging"`.
2. **Connect Phone to PC:** Connect via USB cable; install USB Driver on Windows/PC.
3. **Verify Connection in Eclipse:** Open `DDMS View` in Eclipse to confirm connected device.
4. **Run Application:** Right-click Android project in Eclipse → `Run As → Android Application` → Select connected device from target dialog.

📝 10-MARK ANSWER: EXPLAIN THE COMPLETE ANDROID APPLICATION DEVELOPMENT WORKFLOW IN ECLIPSE WITH ADT, FROM CREATION TO USB DEPLOYMENT.

1. Introduction: Developing Android apps requires setting up IDE, SDK, project structure, and deployment targets.

2. Workflow Flowchart: Draw Install → Configure → Create → Code → Build → Run on Emulator / USB Device.

3. Project Structure: Explain `src`, `gen/R.java`, `res/layout`, `res/values`, and `AndroidManifest.xml`.

4. Code Example: Provide XML layout and Java Activity for Hello World.

5. Deployment: Detail running on AVD Emulator vs deploying on USB hardware device via USB Debugging.

HOW TO WRITE IN EXAM: Draw workflow flowchart → Detail project folder tree → Write Hello World XML and Java code → List 4 USB debugging setup steps.

📌 Quick Revision — Unit III

Eclipse + ADT: Primary IDE environment specified in syllabus.

gen/R.java: Auto-generated resource pointer file; DO NOT EDIT.

AndroidManifest.xml: Core registration file for permissions & activities.

AVD Emulator: Virtual phone configuration for software testing.

USB Debugging: Phone setting required to deploy apps to real hardware.

🧠 MEMORY TRICKS — UNIT III

- Project Folders: **S-G-R-M** → **S**rc, **G**en (R.java), **R**es, **M**anifest.
- USB Deploy Steps: **D-C-V-R** → **D**ebugging on, **C**onnect cable, **V**erify DDMS, **R**un app.

IV Unit IV: User Interface Architecture and Design

Unit Overview:

- Covers Context, Intents, Activity Life Cycle (7 callbacks), Screen Responsiveness, UI Form Widgets, Layout Managers, Menus, and Dialogs.
- Key Exam Drivers: 🔥 Activity Life Cycle Diagram (10M guaranteed), Intents (Explicit vs Implicit), Layout Managers, and Form Controls.

4.1 Application Context & Intent Passing 🔥 Very Important

APPLICATION CONTEXT

Context is an interface granting access to global information about the application environment, system services, and local resource assets.

INTENTS

An **Intent** is an asynchronous messaging object used to request an action from another application component.

Explicit vs Implicit Intents Code Comparison:

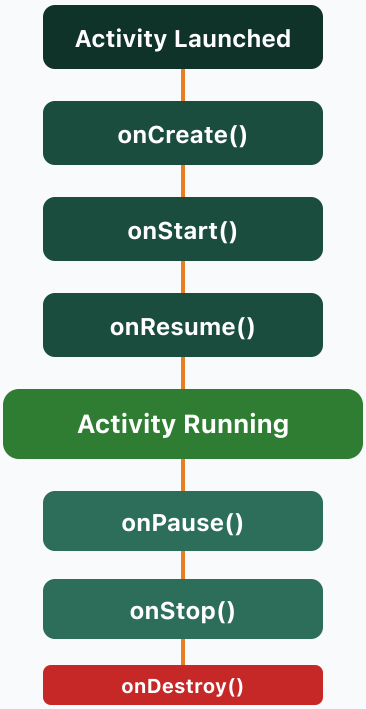
```
// 1. Explicit Intent (Navigating to SecondActivity inside app)
Intent intent1 = new Intent(MainActivity.this, SecondActivity.class);
intent1.putExtra("USERNAME", "Rahul");
startActivity(intent1);

// 2. Implicit Intent (Opening Web Browser externally)
Intent intent2 = new Intent(Intent.ACTION_VIEW, Uri.parse("http://www.jnujaipur.ac.in"));
startActivity(intent2);
```

4.2 Activity Life Cycle (7 States & Transitions) 🔥 Very Important

ACTIVITY DEFINITION

An **Activity** represents a single screen with a user interface. The **Activity Life Cycle** dictates the lifecycle states an activity undergoes from creation to destruction.



Exam Drawing Tip: Draw vertical flowchart showing 7 methods: onCreate → onStart → onResume → [Activity Running] → onPause → onStop → onDestroy.

Explanation of the 7 Lifecycle Callbacks:

1. `onCreate()`: Called when activity is first created. Setup UI view using `setContentView()`.
2. `onStart()`: Called when activity becomes visible to the user.
3. `onResume()`: Called when activity starts interacting with the user (foreground state).
4. `onPause()`: Called when another activity gets focus (partially obscured). Commit unsaved data here.
5. `onStop()`: Called when activity is no longer visible.
6. `onRestart()`: Called when activity is stopped and restarted again before starting.
7. `onDestroy()`: Called before activity is destroyed and memory cleared.

4.3 Screen Sizes, Densities & dp/sp Math ⭐ Important

- **Screen Densities:** `ldpi` (120 dpi), `mdpi` (160 dpi baseline), `hdpi` (240 dpi), `xhdpi` (320 dpi).
- **Density Math Formula:** `px = dp * (dpi / 160)`.
- **Responsive Units:** Use **dp (density-independent pixels)** for layout widths/heights and **sp (scale-independent pixels)** for typography text sizes. Avoid hardcoded **px (pixels)**.

4.4 UI Form Widgets, Controls & Layout Managers 🔥 Very Important

1. Layout Managers:

- **LinearLayout:** Arranges child views linearly in a single `horizontal` or `vertical` orientation. Uses `android:layout_weight` for proportional sizing.
- **RelativeLayout:** Positions child views relative to parent boundaries or sibling views using IDs (e.g., `layout_below`, `layout_toRightOf`).
- **TableLayout:** Arranges views into rows and columns using `TableRow` elements.
- **FrameLayout:** Simple placeholder layout used to hold a single child view or overlapping fragment views.

2. Form Widgets & Java Event Handling Code:

```
// Binding Spinner View with ArrayAdapter in Java
Spinner spinner = (Spinner) findViewById(R.id.spinner1);
String[] courses = { "BCA", "MCA", "B.Tech", "M.Tech" };

ArrayAdapter<String> adapter = new ArrayAdapter<String>(this,
        android.R.layout.simple_spinner_item, courses);
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
spinner.setAdapter(adapter);

// Button Click Handling
Button btnSubmit = (Button) findViewById(R.id.btnSubmit);
btnSubmit.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Toast.makeText(getApplicationContext(), "Form Submitted!", Toast.LENGTH_SHORT).show()
    }
});
```

4.5 Menus, Dialogs & Toast Notifications ⭐ Important

AlertDialog Code Example:

```
AlertDialog.Builder builder = new AlertDialog.Builder(this);
builder.setTitle("Exit App");
builder.setMessage("Are you sure you want to exit?");
builder.setPositiveButton("Yes", new DialogInterface.OnClickListener() {
    public void onClick(DialogInterface dialog, int id) {
        finish();
    }
});
builder.setNegativeButton("No", null);
AlertDialog alert = builder.create();
alert.show();
```

📝 **10-MARK ANSWER: EXPLAIN ACTIVITY LIFE CYCLE WITH A COMPLETE DIAGRAM AND DETAIL ALL 7 CALLBACK METHODS.**

1. Introduction: An Activity is the core UI component. Its lifecycle manages transitions between foreground and background states.

2. Diagram: Draw 7-stage vertical flowchart from `onCreate()` to `onDestroy()`.

3. Method Explanations: Detail all 7 callbacks: `onCreate`, `onStart`, `onResume`, `onPause`, `onStop`, `onRestart`, `onDestroy`.

4. Practical Usage: Explain where to initialize UI (`onCreate`) and where to release heavy resources like GPS/Camera (`onPause`).

5. Conclusion: Proper lifecycle management prevents memory leaks and app crashes during screen orientation changes.

HOW TO WRITE IN EXAM: Define Activity → Draw 7-callback diagram → Write 7 numbered method headings with 2-3 lines each → Mention resource management best practices.

📌 Quick Revision — Unit IV

Explicit Intent: Targets specific class inside same app.

Implicit Intent: Requests action via system intent filter (e.g., open web page).

Activity Callbacks: 7 methods controlling Activity lifecycle.

LinearLayout: Arranges views in horizontal or vertical line.

RelativeLayout: Positions views relative to parent or sibling views.

dp vs sp: Use dp for layout dimensions, sp for text font sizes.

Toast: Transient short feedback notification popup.

🧠 MEMORY TRICKS — UNIT IV

- Activity Life Cycle: **C-S-R-P-S-R-D** → **C**reate, **S**tart, **R**esume, **P**ause, **S**top, **R**estart, **D**estroy.
- Layout Managers: **L-R-T-F** → **L**inear, **R**elative, **T**able, **F**rame.

V Unit V: Database (SQLite)

Unit Overview:

- Covers SQLite Database features, supported data types, SQLiteOpenHelper class (onCreate, onUpgrade), CRUD operations, and Application to Database architecture flow.
- Key Exam Drivers: 🔥 SQLite Features, SQLiteOpenHelper implementation, CRUD Java Code, and Architecture Diagram.

5.1 Understanding SQLite & Supported Data Types 🔥 Very Important

> **SQLITE DEFINITION**
>
> **SQLite** is an open-source, embedded, relational database engine integrated directly into the Android OS. It is serverless, zero-configuration, and transactional.

Supported SQLite Data Types:

- `NULL`: Represents a NULL value.
- `INTEGER`: Signed integer stored in 1, 2, 3, 4, 6, or 8 bytes.
- `REAL`: Floating point value stored as 8-byte IEEE floating point number.
- `TEXT`: Text string stored using database encoding (UTF-8).
- `BLOB`: Binary Large Object data stored exactly as input (e.g., images, files).

5.2 Connecting with Database & SQLiteOpenHelper 🔥 Very Important

> **SQLITEOPENHELPER CLASS**
>
> An abstract helper class used to manage database creation and version management in Android.

Two Core Methods to Override:

1. `onCreate(SQLiteDatabase db)`: Executed when database is created for the first time. Execute `CREATE TABLE` SQL here.
2. `onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion)`: Executed when database version number is incremented. Drop old tables and recreate.

5.3 Complete CRUD Operations Java Code 🔥 Very Important

```
import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

public class DatabaseHelper extends SQLiteOpenHelper {

    private static final String DATABASE_NAME = "StudentDB.db";
    private static final int DATABASE_VERSION = 1;
    private static final String TABLE_NAME = "students";

    public DatabaseHelper(Context context) {
        super(context, DATABASE_NAME, null, DATABASE_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        String CREATE_TABLE = "CREATE TABLE " + TABLE_NAME +
                              "(id INTEGER PRIMARY KEY, name TEXT, email TEXT)";
        db.execSQL(CREATE_TABLE);
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS " + TABLE_NAME);
        onCreate(db);
    }

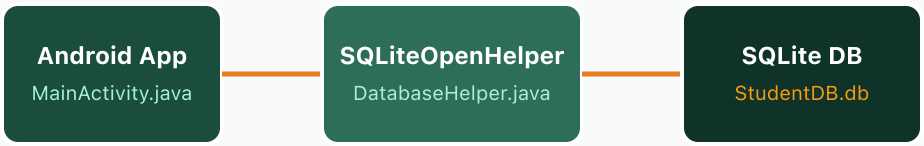
    // 1. INSERT (Create) Data
    public boolean insertData(int id, String name, String email) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues contentValues = new ContentValues();
        contentValues.put("id", id);
        contentValues.put("name", name);
        contentValues.put("email", email);
        long result = db.insert(TABLE_NAME, null, contentValues);
        return result != -1;
    }

    // 2. SELECT (Read) Data
    public Cursor getAllData() {
        SQLiteDatabase db = this.getReadableDatabase();
        return db.rawQuery("SELECT * FROM " + TABLE_NAME, null);
    }
```

```
    // 3. UPDATE Data
    public boolean updateData(int id, String name, String email) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues contentValues = new ContentValues();
        contentValues.put("name", name);
        contentValues.put("email", email);
        int result = db.update(TABLE_NAME, contentValues, "id = ?", new String[]{ String.valu
        return result > 0;
    }

    // 4. DELETE Data
    public Integer deleteData(int id) {
        SQLiteDatabase db = this.getWritableDatabase();
        return db.delete(TABLE_NAME, "id = ?", new String[]{ String.valueOf(id) });
    }
}
```

5.4 Android App to Database Architecture Flow ☆ Important



Exam Drawing Tip: Draw 3 horizontal blocks: Android Application → SQLiteOpenHelper Connection → SQLite Database Engine.

📝 **10-MARK ANSWER: EXPLAIN SQLITE DATABASE IN ANDROID WITH SQLITEOPENHELPER CLASS AND WRITE JAVA CODE FOR DATABASE CREATION, INSERTION, AND QUERYING.**

1. **Introduction:** Android uses embedded SQLite for local structured data storage.
2. **Features:** Serverless, zero-config, single file storage, lightweight, ACID compliant.
3. **SQLiteOpenHelper:** Explain `onCreate()` and `onUpgrade()` methods.
4. **Code Example:** Write complete Java class extending `SQLiteOpenHelper` with `insertData()` and `getAllData()` methods.
5. **Conclusion:** SQLite enables offline data persistence for Android apps.

HOW TO WRITE IN EXAM: List 5 SQLite features → Draw App-to-DB flow diagram → Explain SQLiteOpenHelper → Write complete Java code snippet.

📌 Quick Revision — Unit V

SQLite: Embedded, serverless, zero-config relational database.

SQLiteOpenHelper: Manages DB creation (onCreate) & upgrades (onUpgrade).

ContentValues: Key-value pair object used for insert/update operations.

Cursor: Pointer object returned by query used to iterate result rows.

getWritableDatabase(): Returns DB instance for write/insert operations.

🧠 MEMORY TRICKS — UNIT V

- SQLite Features: **S-Z-S-L-A** → **S**erverless, **Z**ero-config, **S**ingle-file, **L**ightweight, **A**CID.
- CRUD Ops: **C-R-U-D** → **C**reate (Insert), **R**ead (Select), **U**pdate, **D**elete.

📖 Important One-Line Definitions

Term	One-Line Exam Ready Definition
Android OS	An open-source, Linux-based mobile operating system developed by Google and OHA.
Dalvik VM (DVM)	A Java virtual machine optimized for mobile hardware executing Dalvik Executable (.dex) files.
JVM	Java Virtual Machine — an abstract engine executing Java bytecode (.class files).
Inheritance	Mechanism where a child class derives fields and methods from a parent class using `extends`.
Polymorphism	Ability of a method or object to take multiple forms at compile-time or runtime.
Interface	A 100% abstract contract in Java defining method signatures implemented via `implements`.
Abstract Class	A class declared with `abstract` keyword that cannot be instantiated and contains abstract/concrete methods.
Thread	A lightweight concurrent unit of execution within a Java process.
Overloading	Multiple methods in the same class sharing the same name but different parameters.
Overriding	Subclass providing a specific implementation of a superclass method with exact same signature.
Activity	An Android component representing a single screen with a user interface.
Activity Lifecycle	The series of state callbacks (onCreate to onDestroy) an Activity transitions through.
Context	Interface granting access to application resources, environment data, and system services.
Intent	An asynchronous messaging object used to request an action from another component.
Explicit Intent	An intent explicitly specifying the target component class name.
Implicit Intent	An intent specifying a general action letting the OS find matching target apps.
Widget	A visual UI control element on screen (e.g., Button, TextView, EditText).

Layout	A container object controlling the positioning and alignment of child views.
Spinner	A dropdown widget allowing selection of a single item from a list.
AlertDialog	A popup modal window displaying alerts, messages, and action buttons.
Toast	A transient pop-up message showing short feedback on screen without blocking user input.
SQLite	An embedded, serverless, relational database engine integrated inside Android.

⚖️ Important Comparison Tables

1. Android OS vs. Traditional OS (Windows/Linux)

Feature	Android OS	Traditional OS
Target Hardware	Mobile touchscreen devices (Smartphones, Tablets).	Desktop PCs, Laptops, Servers.
Execution Engine	Dalvik / ART VM executing `.dex` files.	Native binary code / JVM executing `.class` files.
Power Management	Aggressive, mobile-optimized power saving.	Standard continuous power management.
UI Paradigm	Touchscreen gestures, virtual keyboard.	Mouse, Keyboard, Monitor.

2. JVM vs. DVM (Dalvik Virtual Machine)

Feature	JVM (Java Virtual Machine)	DVM (Dalvik Virtual Machine)
Executable File	Executes Java Bytecode (`.class` files).	Executes Dalvik Executable (`.dex` files).
Architecture	Stack-based architecture.	Register-based architecture (faster on mobile).
Memory Footprint	Requires higher memory & CPU.	Optimized for low RAM & mobile processor constraints.
File Packaging	Multiple `.class` files.	Combines multiple `.class` files into single `.dex`.

3. Explicit Intent vs. Implicit Intent

Feature	Explicit Intent	Implicit Intent
Target Specification	Specifies exact component class name.	Specifies general action (e.g., ACTION_VIEW).
Use Case	Launching activities inside *same application*.	Launching components across *external apps* (Browser, Dialer).
Example	`new Intent(this, ProfileActivity.class)`	`new Intent(Intent.ACTION_VIEW, uri)`

4. Button vs. ToggleButton

Feature	Standard Button	ToggleButton
State	Single push action (stateless).	Dual state (ON / OFF toggle).
Visual Indicator	Simple label text.	Includes light indicator bar showing state.
Use Case	Triggering immediate action (Submit, Login).	Toggling setting state (Wi-Fi On/Off, Sound Mute).

5. SQLite vs. Traditional Database (MySQL/Oracle)

Feature	SQLite Database	Traditional RDBMS (MySQL/Oracle)
Architecture	Serverless, embedded in application process.	Client-Server architecture.
Configuration	Zero-configuration required.	Requires server setup & user privileges administration.
File Storage	Entire DB stored in single local file.	Multiple system data files managed by database server.
Target Usage	Local mobile persistent storage.	Enterprise web applications with concurrent remote users.

🎯 Master Exam Revision

Exam Master Guide: Review high-priority 🔥 topics right before entering the exam room. Practice Java code snippets and lifecycle diagrams.

A. Top Priority Topic Hierarchy

Unit	🔥 Must-Know Topics
Unit I	Android 5-Layer Architecture Diagram · Features of Android OS · ADT & SDK Tools
Unit II	Overloading vs Overriding · Interface vs Abstract Class · Thread Lifecycle · JVM Architecture · 7 Java Programs
Unit III	Android Development Workflow · Eclipse ADT Project Structure · Hello World Code · USB Debugging Deploy Steps
Unit IV	Activity Life Cycle (7 callbacks + diagram) · Explicit vs Implicit Intents · Layout Managers · Form Controls
Unit V	SQLite Features · SQLiteOpenHelper (onCreate/onUpgrade) · CRUD Java Code · App-to-DB Flow Diagram

📅 One-Day Revision Sheet

Compressed summary for fast review 2 hours before the exam.

UNIT I: Introduction & Architecture

- Android: Open-source, Linux-based mobile OS created by OHA / Google.
- 5 Architecture Layers: Applications, Framework, Libraries/Runtime, HAL, Kernel.
- Linux Kernel: Handles memory, process, security & hardware drivers.
- Dalvik VM: Executes .dex files optimized for low RAM mobile hardware.
- ADB: Command-line tool connecting PC to emulators or USB devices.

UNIT II: OOP in Java & JVM

- Encapsulation = Hiding fields; Abstraction = Hiding implementation.
- Inheritance uses extends; Interface uses implements.
- Overloading: Same method name, different args in same class.
- Overriding: Subclass redefines superclass method signature.
- JVM Memory: Heap (Objects), Stack (Local vars), Method Area (Classes).
- Thread States: New → Runnable → Running → Blocked → Terminated.

UNIT III: Development Tools

- Eclipse + ADT: Official IDE setup specified in syllabus.
- gen/R.java: Auto-generated resource ID pointer; DO NOT EDIT.
- AndroidManifest.xml: Core registration file for permissions & activities.
- AVD: Virtual Android emulator configuration for software testing.
- USB Debugging: Phone setting required to run apps on hardware.

UNIT IV: UI Architecture & Widgets

- Context: Interface providing access to app resources &
- Explicit Intent: Targets specific class inside same app.

services.

Implicit Intent: Requests action via system intent filter.

Activity Life Cycle: onCreate → onStart → onResume → onPause → onStop → onDestroy.

Layouts: LinearLayout (Line), RelativeLayout (Relative positioning).

Toast: Short transient feedback notification popup.

UNIT V: Database (SQLite)

SQLite: Embedded, serverless, zero-config relational database.

SQLiteOpenHelper: Manages DB creation (onCreate) & upgrades (onUpgrade).

ContentValues: Key-value pair container used for insert/update operations.

Cursor: Pointer object returned by rawQuery to iterate rows.

? Expected Practice Question Bank

UNIT I QUESTIONS

1. What is Android? (2M)
2. List 4 features of Android OS. (2M)
3. Explain Android Development Tools. (5M)
4. Differentiate JVM and DVM. (5M)
5. Explain 5-layer Android Architecture with diagram. (10M)

UNIT II QUESTIONS

1. Define Inheritance in Java. (2M)
2. What is a Thread? (2M)
3. Compare Overloading and Overriding. (5M)
4. Compare Interface and Abstract Class. (5M)
5. Explain JVM Architecture in detail with diagram. (10M)

UNIT III QUESTIONS

1. What is AVD? (2M)
2. What is R.java? (2M)
3. Explain Android Project Folder Structure. (5M)
4. Explain USB Debugging deployment steps. (5M)
5. Explain Android Development Workflow in Eclipse with ADT. (10M)

UNIT IV QUESTIONS

1. What is an Activity? (2M)
2. Define Intent. (2M)
3. Compare Explicit and Implicit Intents. (5M)
4. Explain LinearLayout vs RelativeLayout. (5M)
5. Explain Activity Life Cycle with diagram and callbacks. (10M)

UNIT V QUESTIONS

1. What is SQLite? (2M)
2. What is SQLiteOpenHelper? (2M)
3. Explain features of SQLite database. (5M)
4. Explain Cursor and ContentValues. (5M)
5. Explain SQLite integration in Android with Java code. (10M)

GENERAL STRATEGY

1. Draw clean diagrams for 10M answers.
2. Write complete Java code blocks.
3. Use comparison tables for differences.
4. Manage exam time: 10M = 20 mins max.

📝 Topper Answer-Writing Strategy

How to Score Maximum Marks in Android Programming:

1. **Standard Definitions First:** Always start 5-mark and 10-mark answers with a formal definition box.
2. **Draw Clean Diagrams:** Always draw diagrams for Android Architecture, JVM Architecture, Activity Lifecycle, and App-to-DB flow. A neat diagram earns 2-3 immediate marks.
3. **Write Exam-Friendly Java Code:** Use clean indentation, standard class names, and comments for Java programs. Always mention expected output.
4. **Use Comparison Tables:** Whenever asked for differences (e.g., Overloading vs Overriding, JVM vs DVM, Interface vs Abstract Class), use tabular format.
5. **Structure 10-Mark Answers Cleanly:** Use this proven topper template:
   Introduction → Definition → Diagram → Detailed Points → Code Snippet → Advantages → Conclusion.
6. **Highlight Keywords:** Underline or bold essential terms (e.g., **.dex**, **R.java**, **SQLiteOpenHelper**, **onCreate()**).

ALL THE BEST FOR YOUR EXAM! 🌟

Android Programming · BCA Semester V · Paper BCAAPCO505T22

Jaipur National University, Jaipur · Complete Topper Notes